MEGATHON X: SINGULARITY – M1 APPROACH (DUAL-TRACK SUBMISSION)

Team: PPAP | **Team ID: 320** | **Problem Statement:** Accessible AI for All (Bhashini) | Languages: English, Hindi, Telugu

Executive Summary (Dual-Track Submission): Team PPAP is submitting dual candidate architectures for the Bhashini challenge to address Indic input and output gaps across **Visual (Mandatory)**, **Ageing**, and **Hand-Motor** conditions: **Page 1** details **SunoShop (Use Case 2: eCommerce)**, a voice-first shopping proxy with offline review RAG and entity masking; **Page 2** details **Oriel (Use Case 3: City Signage)**, a zero-tap multimodal sign reader with multi-frame verification. Presenting both allows parallel Phase 1 empirical benchmarking before converging for M2.

1 1. Track 1: SunoShop (Use Case 2 – eCommerce) – Problem Statement & Understanding

Standard AI assistants assume users can see, hear, type, and read English. Modern eCommerce breaks this across two critical dimensions:

- **Input Gap:** Blind and motor-impaired individuals cannot parse dense DOMs, hidden controls, or drag-and-drop sliders (WCAG 2.5.7). Screen readers force them to listen to hundreds of unlabelled elements in an exhaustive linear stream.
- **Output Gap:** Assistants emit lengthy visual descriptions and hallucinate product specifications, while translation corrupts brand names and prices into phonetic gibberish. Users remain unprotected from dark patterns like hidden cart fees.
- **Target Conditions:** Visual (Mandatory / blind-first), Ageing, and Hand-motor, with zero reliance on specialized hardware.

2 2. Idea

SunoShop is a web application acting as an accessible voice-first shopping proxy built on a *ground-then-speak* paradigm:

- Speaks only facts verified within an offline-cached Product Knowledge Object (PKO), strictly separating unstructured reviews from factual specs.
- Asserts "the specs state" only for confirmed attributes, actively blocks checkout dark patterns, and communicates via Bhashini Indic models.
- Confines operations strictly to shopping assistance, deliberately excluding ungrounded medical or financial advice.

3 3. Approach and Architecture

Computation is tiered: repetitive tasks run offline on-device; cloud calls are reserved strictly for grounded reasoning.

1. **Offline Ingestion:** Catalogs are normalized into PKO JSONs. Gemini Flash Vision pre-computes image descriptions, and OCR structures size charts offline to eliminate live-vision latency.
2. **Triggering:** Voice activity detection (VAD) captures speech hands-free, with a massive push-to-talk button as an alternative.
3. **Fast-Path Routing & Entity Masking (Rule-Based):** Intercepts simple commands (*repeat, slower, simpler*). Brand names, prices, and numbers are masked with tokens prior to translation to prevent transliteration errors.
4. **Translation Pipeline:** Bhashini ASR transcribes voice to text, translated to English pivot text (≤ 50 words).
5. **Grounded Reasoning:** Gemini Flash-Lite retrieves chunked reviews via aspect cosine-similarity (RAG) and exact specs via direct PKO lookup.
6. **Deterministic Verification:** A finite state machine (not an LLM) controls checkout, enforcing explicit spoken confirmation while stripping hidden fees.
7. **Synthesis & Accessible Rendering:** Chained Bhashini Translation+TTS restores unmasked entities. Outputs spatial audio via Web Audio API alongside synchronized on-screen captions.

Condition	Input Gap Feature	Output Gap Feature
Visual	Semantic headings; Bhashini voice commands; native TalkBack/NVDA coexistence	Key fact first; spatial audio (panning results left-to-right); VLM image alt-text; grounded PKO summaries
Ageing	Large single button; explicit "explain simpler" voice trigger; generous timeouts	One fact per screen in large text; chunked speech with pauses; claim-vs-experience verification
Hand-motor	Voice-driven slider bypass (WCAG 2.5.7); zero-touch speech auto-capture	High-contrast interface with massive auto-enlarged tap targets

Condition Inference & Fallbacks: Inferred from interaction friction in first 20–30s (repeated ASR timeouts, "explain again" requests) to suggest adaptations without unconsented context switches. **Fallback Ladder:** Bhashini → local IndexedDB PKO cache + device browser TTS. **Stack:** React + Vite PWA, FastAPI, Gemini API, Bhashini (ASR, NMT, TTS), Web Audio API, IndexedDB Service Worker.

4 4. Planned Testing

Test Set: 30–40 real products in PKO format in Telugu, Hindi, English, covering complex size charts, code-mixed reviews, and 15 planted prompt-injection reviews. **Metrics:** Time-to-information; words listened; hallucination rate (0%); prompt-injection bypasses (0); false-accept rate on checkout confirmation; brand/spec translation preservation. **Baselines:** Raw eCommerce DOM via NVDA/TalkBack; plain LLM responses through Bhashini TTS. **Validation:** 3–5 participants across five shopping tasks (including blindfolded teammates and native TalkBack/NVDA users). **Boundaries:** Cached catalog snapshot, simulated checkout, heuristic condition inference.

MEGATHON X: SINGULARITY – M1 APPROACH (DUAL-TRACK SUBMISSION)

Team: PPAP | **Team ID: 320** | **Problem Statement:** Accessible AI for All (Bhashini) | Languages: English, Telugu, Hindi

Dual-Track Submission Note (Track 2): Complementing Page 1's digital eCommerce layer, **Page 2** presents **Oriel**, addressing physical urban navigation under the same Bhashini multilingual backbone. Both tracks share on-device fast-path routing, zero-tap triggers, and offline fallbacks to maximize coverage across indoor commerce and outdoor mobility.

5 1. Track 2: Oriel (Use Case 3 – City Signage) – Problem Statement & Understanding

Physical city streets amplify assistive failures across visual and linguistic boundaries:

- **Input Gap:** Blind and motor-impaired individuals must aim cameras at unseen targets and tap tiny buttons; 51.6% of photos taken by blind users exhibit critical quality flaws, predominantly framing and motion blur (VizWiz, CVPR 2020).
- **Output Gap:** Multimodal models emit ungrounded visual descriptions and struggle with small, stylized, or glared street text and Indian scripts (NetraLink, ICVGIP 2025). Street signs frequently mix Telugu, Hindi, and English, where a single misidentified route number misdirects the user.
- **Target Conditions:** Visual (Mandatory / blind-first), Ageing, and Hand-motor, with zero reliance on specialized hardware.

6 2. Idea

Oriel is a zero-tap web application reading city signs aloud based on an active *verify-then-guide* paradigm:

- Speaks only facts verified across multiple sharp frames (route, destination, platform, arrow), marking unconfirmed text as uncertain.
- Actively coaches aiming when a critical fact is obscured: *"I can read Secunderabad but not the route number. Hold still, zooming in."*
- Asserts "the sign says" strictly for confirmed text via Bhashini Indian language models, restricting scope to sign reading (no ungrounded route planning).

7 3. Approach and Architecture

Repetitive filtering executes locally on-device; cloud pipelines are engaged only for sharp, validated text frames.

1. **Continuous Monitoring (On-Device):** Streams low-resolution camera frames, tilt, and motion sensors; optional MediaPipe head tracking.
2. **Triggering:** Auto-captures upon steady, text-bearing frames; hands-free voice command ("read"); head dwell; or a single massive button.
3. **Quality Gate & Active Guidance (Rule-Based):** Detects invalid tilt (facing ground/sky), blur, darkness, or glare. Spoken guidance runs in bounded 5-second sweeps (*"Slowly turn left, I'll say stop"*); caps at 3 attempts before honest abstention.
4. **High-Resolution Burst:** Captures a 1-second burst, selects the 3 sharpest frames, and crops around text to defeat hand tremors.
5. **Multimodal Reading:** Gemini Flash-Lite parses structured JSON per frame (type, script, route, destination, arrows); validated against PaddleOCR.
6. **Multi-Frame Verification:** A field is confirmed only when 2 of 3 frames agree and Unicode script is valid; otherwise triggers re-read or abstains.
7. **Speech Synthesis & Rendering:** Feeds condition-tailored views. Chained Bhashini Translation+TTS locks route numbers and place names against transliteration corruption; provides synchronized on-screen captions and a two-way speech relay ("Ask someone nearby").

Condition	Input Gap Feature	Output Gap Feature
Visual	Guided aiming; Bhashini voice commands; nod confirmation	Key fact first, "more"/"spell it"; confirmed facts vs "it looks like"; adjustable speech rate
Ageing	Progressive help ladder; long timeouts; single large button	One fact per screen in large text; chunked speech with pauses; "explain simpler" option
Hand-motor	Zero-tap auto-capture; burst against tremor; head-dwell targets	High-contrast interface with massive auto-enlarged dwell targets; full voice control

Condition Inference & Fallbacks: Inferred from initial 20–30 seconds of interaction friction to suggest adaptations without silent switching. On-device caching caps reads at 3 per sign. **Privacy & Fallback:** Captured frames remain ephemeral with faces blurred; offline fallback via Tesseract.js. **Stack:** React + Vite PWA, FastAPI, Gemini API, Bhashini (ASR, NMT, TTS), MediaPipe Tasks Vision, IndexedDB Service Worker.

8 4. Planned Testing

Test Set: 30–40 real signs photographed in Hyderabad in Telugu, Hindi, and English (bus stops, metro, street, shops), spanning clear, blurry, far, cut-off, and glare conditions with 2–3 frames per sign and ground truth labels. **Metrics:** Detection rate; key-fact accuracy; **wrong-but-confident rate** (headline safety metric); correct abstention rate; time-to-key-fact; taps per reading (target: 0); bad-aim recovery rate; route/place name translation preservation; end-to-end latency. **Baselines:** Plain LLM response read via Bhashini TTS; single-frame baseline; generic "try again" feedback. **Validation:** 3–5 participants across five sign tasks (including blindfolded teammates and simulated motor constraints); live outdoor bus stop testing. **Boundaries:** Stationary use only (no dynamic route planning), reduced aiming dependency, mobile data dependency.